

Computational Learning

Goal of Learning Theory

To understand

- What kinds of tasks are learnable?
- What kind of data is required for learnability?
- What are the (space, time) requirements of the learning algorithm ?

To develop and analyze models:

- Develop algorithms that provably meet desired criteria
- Prove guarantees for successful algorithms.

we will focus not on individual learning algorithms, but rather on broad classes of learning algorithms characterized by the hypothesis spaces they consider, the presentation of training examples, etc.

Our goal is to answer questions such as:

Sample complexity: How many training examples are needed for a learner to converge (with high probability) to a successful hypothesis?

Computational complexity: How much computational effort is needed for a learner to converge (with high probability) to a successful hypothesis?

Mistake bound (Error): How many training examples will the learner misclassify before converging to a successful hypothesis?

what it means for the learner to be "successful".

The learner must output a hypothesis identical to the target concept. Alternatively, we might simply require that it output a hypothesis that agrees with the target concept most of the time, or that it usually output such a hypothesis.

$$h(x) = c(x)$$

where h belongs to H space and c is concept over x

Similarly, we must specify how training examples are to be obtained by the learner.

We might specify that training examples are presented by a helpful teacher, or obtained by the learner performing experiments, or simply generated at random according to some process outside the learner's control.

As we might expect, the answers to the above questions depend on the particular setting, or learning model, we have in mind.

we consider a particular setting for the learning problem, called the **probably approximately correct (PAC) learning model**. We begin by specifying the problem setting that defines the PAC learning model, then consider the questions of **how many training examples** and **how much computation** are required in order to learn various classes of target functions within this PAC model.

PAC Model: (Probably Approximately Correct)

For the sake of simplicity ,

Learn the boolean values concepts from Noise free training data.

Some can be extended to learning from certain types of noisy data.

The Problem Setting :

Let X is the set of all possible instances over which target functions may be defined.

For example,

X represent the set of all people, each described by the attributes age (e.g., young or old) and height (short or tall).

Let C refer to some set of target concepts that our learner called upon to learn.

Each target concept c in C corresponds to some subset of X , or equivalently to some boolean-valued function $c: X \rightarrow \{0, 1\}$.

For example, one target concept c in C be the concept "people who are skiers."

If x is a positive example of c , then we will write $c(x) = 1$;
if x is a negative example, $c(x) = 0$.

Assume instances are generated at random from X according to some probability distribution D .

In general, D may be any distribution, and it will not generally be known to the learner. All that we require of D is to be stationary i.e Distribution not change over time.

Training examples are generated by drawing an instance x at random according to D , then presenting x along with its target value, $C(x)$ to the learner.

The learner L considers some set H of possible hypotheses when attempting to learn the target concept.

For example, H might be the set of all hypotheses describable by conjunctions of the attributes **age** and **Height**. After observing a sequence of training examples of the target concept C , L must output some hypothesis h from H , which is its estimate of c .

We evaluate the success of L by the performance of h over new instances drawn randomly from X according to D , the same probability distribution used to generate the training data.

Error of a Hypothesis

Let us begin by defining the **true error** of a hypothesis h with respect to target concept c and instance distribution D . Informally, the true error of h is just the error rate we expect when applying h to future instances drawn according to the probability distribution D .

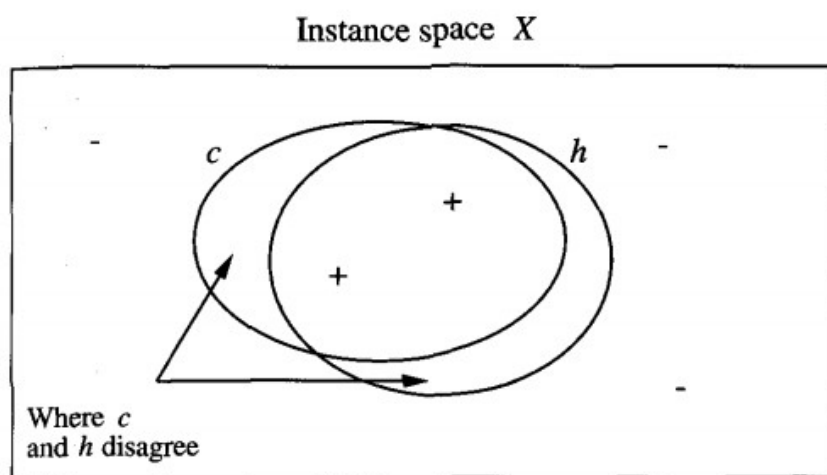
Definition: The **true error** (denoted $error_D(h)$) of hypothesis h with respect to target concept c and distribution D is the probability that h will misclassify an instance drawn at random according to D .

$$error_D(h) = \Pr_{x \in D} [c(x) \text{ not equal to } h(x)]$$

Or

$$error_D(h) \equiv \Pr_{x \in D} [c(x) \neq h(x)]$$

True error shown in graphical representation as below



The error of hypothesis h with respect to target concept c . The error of h with respect to c is the probability that a randomly drawn instance will fall into the region where h and c disagree on its classification. The $+$ and $-$ points indicate positive and negative training examples. Note h has a nonzero error with respect to c despite the fact that h and c agree on all five training examples observed thus far.

The error region to be small where h and c disagree on instance space.

PAC Learnability

Our aim is to characterize classes of target concepts that can be reliably learned from a reasonable number of randomly drawn training examples and a reasonable amount of computation.

What kinds of statements about learnability should we guess hold true?

We might try to characterize the number of training examples needed to learn a hypothesis h for which $\text{error}_D(h) = 0$.

Unfortunately, it turns out this is futile(not useful) in the setting we are considering, **for two reasons**.

- **First**, unless we provide training examples corresponding to every possible instance in X (an unrealistic assumption),

There may be multiple hypotheses consistent with the provided training examples, and the learner cannot be certain to pick the one corresponding to the target concept.

- **Second**, given that the training examples are drawn randomly,

There will always be some nonzero probability that the training examples encountered by the learner will be misleading.

To accommodate these two difficulties, we weaken our demands on the learner in two ways.

- **First**, we will not require that the learner output a zero error hypothesis-

we will require only that its error be bounded by some constant, c , that can be made arbitrarily small.

- **Second**, we will not require that the learner succeed for every sequence of randomly drawn training examples

-we will require only that its probability of failure be bounded by some constant, δ , that can be made arbitrarily small.

In short, we require only that the learner probably learn a hypothesis that is approximately correct-hence the term probably approximately correct learning.

Consider some class C of possible target concepts and a learner L using hypothesis space H . Loosely speaking, we will say that the concept class C is PAC-learnable by L using H if, for any target concept c in C , L will with probability $(1 - \epsilon)$ output a hypothesis h with $\text{error}_D(h) < \epsilon$, after observing a reasonable number of training examples and performing a reasonable amount of computation.

More precisely,

Definition: Consider a concept class C defined over a set of instances X of length n and a learner L using hypothesis space H . C is **PAC-learnable** by L using H if for all $c \in C$, distributions $\mathcal{D}$ over X , ϵ such that $0 < \epsilon < 1/2$, and δ such that $0 < \delta < 1/2$, learner L will with probability at least $(1 - \delta)$ output a hypothesis $h \in H$ such that $\text{error}_{\mathcal{D}}(h) \leq \epsilon$, in time that is polynomial in $1/\epsilon$, $1/\delta$, n , and $\text{size}(c)$.

Our definition requires two things from L .

First, L must, with arbitrarily high probability $(1 - \delta)$, output a hypothesis having arbitrarily low error (ϵ) .

Second, it must do so efficiently-in time that grows at most polynomially with $1/\epsilon$ and $1/\delta$.

Sample Complexity for Finite Hypothesis Spaces

- The growth in the number of required training examples with problem size is called the **sample complexity** of the learning problem.
- We will consider only **consistent learners**, which are those that maintain a training error of 0.
- We can derive a bound on the number of training examples required by any consistent learner!
- **Fact:** Every consistent learner outputs a hypothesis belonging to the version space.

$$VS_{H,D} = \{h \text{ belongsto } H \mid (\text{forall}(x, c(x)) \text{ belongsto } D) (h(x)=c(x))\}$$

- The reason is, by the definition the version space $VS_{H,D}$ contains every consistent hypothesis in H .
- Therefore, we need to bound the number of examples needed to assure that the version space contains no unacceptable hypothesis.

Definition: Consider a Hypothesis space H , target concept c , instance distribution D , and set of training examples D of c . The version space $VS_{H,D}$ is said to be **ϵ -exhausted** with respect to c and D , if every hypothesis h in $VS_{H,D}$ has error less ϵ with respect to c and D .

$$(\text{forall } h \text{ belongsto } VS_{H,D}) \text{ error}_D(h) < \epsilon$$

The version space is **ϵ -exhausted** just in the case that all hypotheses consistent with the observed training examples that is those hypothesis with zero training error happen to have true error less than ϵ .

To Summarize,

Sample Complexity: Finite Hypothesis Spaces

Consistent Case

$$m \geq \frac{1}{\epsilon} \left[\ln(|H|) + \ln\left(\frac{1}{\delta}\right) \right]$$

Labeled examples are sufficient .

Inconsistent Case

What if there is no perfect h ?

Theorem: After m examples with probability $\geq 1 - \delta$, all $h \in H$ have $err_D(h) - err_S(h) < \epsilon$, for

$$m \geq \frac{2}{\epsilon^2} \left[\ln(|H|) + \ln\left(\frac{2}{\delta}\right) \right]$$

Sample Complexity Example

$\mathcal{C}$: Conjunction of n Boolean literals. Is $\mathcal{C}$ PAC-learnable?

$$|\mathcal{H}| = 3^n$$

$$m \geq \frac{1}{\epsilon} (n \ln 3 + \ln(1/\delta))$$

Concrete examples:

- $\delta=\epsilon=0.05$, $n=10$ gives 280 examples
- $\delta=0.01$, $\epsilon=0.05$, $n=10$ gives 312 examples
- $\delta=\epsilon=0.01$, $n=10$ gives 1,560 examples
- $\delta=\epsilon=0.01$, $n=50$ gives 5,954 examples

Result holds for any consistent learner, such as FindS

Thank you